

bytes.Buffer

Hướng dẫn sử dụng đầy đủ trong Go

Package: bytes | Go Standard Library

Tổng quan

bytes.Buffer là một buffer đọc/ghi linh hoạt cho dữ liệu dạng byte. Nó implement cả **io.Reader** và **io.Writer**, cho phép vừa ghi vừa đọc dữ liệu trong cùng một buffer.

Khởi tạo

```
// Cách 1: Zero value – dùng được luôn không cần khởi tạo
var buf bytes.Buffer

// Cách 2: bytes.NewBuffer
buf := bytes.NewBuffer([]byte("hello"))

// Cách 3: bytes.NewBufferString
buf := bytes.NewBufferString("hello")
```

Ghi dữ liệu vào Buffer

```
var buf bytes.Buffer

buf.Write([]byte("hello ")) // ghi []byte
buf.WriteString("world")   // ghi string
buf.WriteByte('\n')        // ghi 1 byte
buf.WriteRune('□')         // ghi 1 rune (UTF-8)

fmt.Fprintf(&buf, "số: %d", 42) // ghi có format
```

Đọc dữ liệu từ Buffer

```
buf := bytes.NewBufferString("hello world\nline2")

// Xem nội dung – KHÔNG tiêu thụ (con trỏ đọc không dịch chuyển)
data := buf.Bytes() // trả []byte
str := buf.String() // trả string

// Đọc có tiêu thụ – con trỏ đọc tiến lên
p := make([]byte, 5)
n, _ := buf.Read(p) // đọc tối đa len(p) byte
b, _ := buf.ReadByte() // đọc 1 byte
r, _, _ := buf.ReadRune() // đọc 1 rune
```

```
line, _ := buf.ReadString('\n') // đọc đến delimiter → string
line, _ := buf.ReadBytes('\n') // đọc đến delimiter → []byte
```

❑ Bytes() và String() chỉ XEM dữ liệu. Dùng các phương thức Read* để TIÊU THỤ (con trỏ đọc tiến lên).

Reset & Tái sử dụng

```
buf.Reset() // xóa nội dung, giữ lại bộ nhớ đã alloc → hiệu quả khi tái dùng
buf.Truncate(n) // cắt còn n byte đầu
buf.Grow(n) // pre-alloc thêm n byte, tránh realloc nhiều lần
```

Bảng tổng hợp các Method

Method	Mô tả	Ghi chú
Write(p []byte)	Ghi slice byte vào buffer	❑ Tiêu thụ
WriteString(s string)	Ghi string vào buffer	❑ Tiêu thụ
WriteByte(c byte)	Ghi 1 byte	❑ Tiêu thụ
WriteRune(r rune)	Ghi 1 rune (UTF-8)	❑ Tiêu thụ
Read(p []byte)	Đọc tối đa len(p) byte	❑ Tiêu thụ
ReadByte()	Đọc 1 byte	❑ Tiêu thụ
ReadRune()	Đọc 1 rune	❑ Tiêu thụ
ReadString(delim)	Đọc đến delimiter, trả string	❑ Tiêu thụ
ReadBytes(delim)	Đọc đến delimiter, trả []byte	❑ Tiêu thụ
Bytes()	Trả toàn bộ nội dung dưới dạng []byte	❑ Chỉ xem
String()	Trả toàn bộ nội dung dưới dạng string	❑ Chỉ xem
Len()	Số byte chưa đọc	❑ Chỉ xem
Cap()	Dung lượng hiện tại	❑ Chỉ xem
Reset()	Xóa nội dung, giữ bộ nhớ	
Truncate(n)	Cắt còn n byte đầu	
Grow(n)	Pre-alloc thêm n byte	

Use Cases phổ biến

1. Build string hiệu quả (thay cho +=)

```
var sb bytes.Buffer
for i := 0; i < 1000; i++ {
```

```

    fmt.Fprintf(&sb, "item %d\n", i)
}
result := sb.String()

```

❑ Dùng `bytes.Buffer` thay cho `+=` trong vòng lặp giúp tránh tạo nhiều string tạm, cải thiện hiệu năng đáng kể.

2. Dùng làm `io.Writer` / `io.Reader`

```

// Capture output của hàm nhận io.Writer
var buf bytes.Buffer
json.NewEncoder(&buf).Encode(myStruct) // ghi JSON vào buffer

// Dùng buffer như io.Reader
json.NewDecoder(&buf).Decode(&out) // đọc JSON từ buffer

```

3. Kết hợp với `io.Copy`

```

var buf bytes.Buffer
io.Copy(&buf, resp.Body) // copy toàn bộ HTTP response vào buffer
fmt.Println(buf.String())

```

4. Tái sử dụng với `sync.Pool`

```

var pool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

buf := pool.Get().(*bytes.Buffer) // lấy buffer từ pool
buf.Reset() // bắt buộc Reset trước khi dùng
defer pool.Put(buf) // trả lại pool khi xong

buf.WriteString("data...")

```

❑ Dùng `sync.Pool` giúp tái sử dụng buffer, giảm áp lực GC trong các ứng dụng high-throughput.

So sánh: `bytes.Buffer` vs `strings.Builder`

Tính năng	<code>bytes.Buffer</code>	<code>strings.Builder</code>
Output	[]byte / string	string
Đọc dữ liệu (Read*)	✓ Có	✗ Không
<code>io.Reader</code>	✓ Có	✗ Không
<code>io.Writer</code>	✓ Có	✓ Có
Mục đích chính	Buffer đọc/ghi 2 chiều	Chỉ build string

Nguyên tắc lựa chọn:

- Chỉ ghép chuỗi → dùng `strings.Builder` (nhanh hơn, đơn giản hơn)
- Cần đọc/ghi hoặc tương tác với `io.*` → dùng `bytes.Buffer`
- Hiệu năng cao + tái sử dụng nhiều → kết hợp với `sync.Pool`